

# PROJECT DOCUMENTATION

## Online Examination System Backend API

**Submitted By:** Ankit Tripathi

**Role:** Backend Developer

**Tech Stack:** NestJS, TypeScript, PostgreSQL, Prisma, Docker, PDFKit, Winston

**Date:** August 2026

*(A highly scalable, secure, and automated backend system for modern educational institutions and competitive platforms)*

## 1. Problem Statement

Educational institutions require robust, secure, and scalable backend systems to conduct and evaluate online assessments efficiently. The objective of this project is to develop an end-to-end backend API for an Online Examination System that digitalizes the entire assessment lifecycle with strict anti-cheat and automated grading mechanisms.

### Core Objectives:

- **Role-Based Access Control:** Provide distinct, strictly guarded functionalities for three user roles: Admins, Instructors, and Students.
- **Complete Examination Lifecycle:** Handle course creation, question bank management (MCQs and Coding questions), exam scheduling, secure attempts, and instant result publishing.
- **Evaluation & System Integrity:** Ensure exact automated grading with configurable negative marking, prevent session hijacking, and maintain tamper-proof server-side timers.

- **Production Readiness:** Deliver a backend that includes Winston-based request logging, global exception handling, dynamic PDF scorecard generation, comprehensive unit testing, and Docker containerization.

## 2. Technology Stack

The project utilizes a modern, enterprise-grade technology stack chosen for its type safety, modular architecture, and high performance.

| Category        | Technology        | Justification                                                                                                     |
|-----------------|-------------------|-------------------------------------------------------------------------------------------------------------------|
| Language        | TypeScript        | Provides static typing, reducing runtime errors and improving code maintainability.                               |
| Framework       | NestJS            | Offers a strict modular architecture, ideal for enterprise scalability and dependency injection.                  |
| Database        | PostgreSQL        | A highly reliable relational database, perfect for handling strict assessment constraints and relational records. |
| ORM             | Prisma            | Provides type-safe database access, automated migrations, and excellent developer ergonomics.                     |
| Authentication  | Passport.js (JWT) | Stateless, scalable authentication mechanism for secure API access.                                               |
| File Generation | PDFKit            | Dynamically generates official, downloadable student scorecards in PDF format.                                    |

|                         |         |                                                                                            |
|-------------------------|---------|--------------------------------------------------------------------------------------------|
| <b>Logging</b>          | Winston | Advanced logging system recording activities to the console and dedicated error log files. |
| <b>Containerization</b> | Docker  | Ensures environment consistency across development, testing, and production workflows.     |

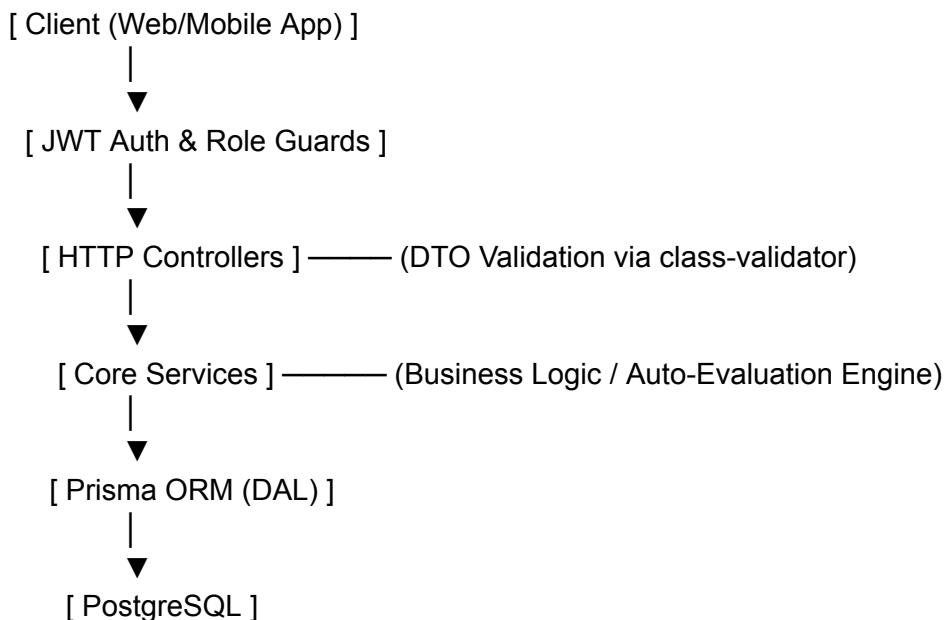
### 3. Project Architecture

The system is built on a Modular RESTful Architecture utilizing NestJS's dependency injection and domain-driven design.

#### 3.1. Architectural Request Flow

Below is the request lifecycle demonstrating how data flows through the application securely:

Plaintext



#### 3.2. Cross-Cutting Concerns

- **Global Logging:** Winston logger automatically records every incoming request, bootstrap event, and unhandled exception.
- **Global Exception Filter:** Catches errors and formats them into a standardized JSON response structure.

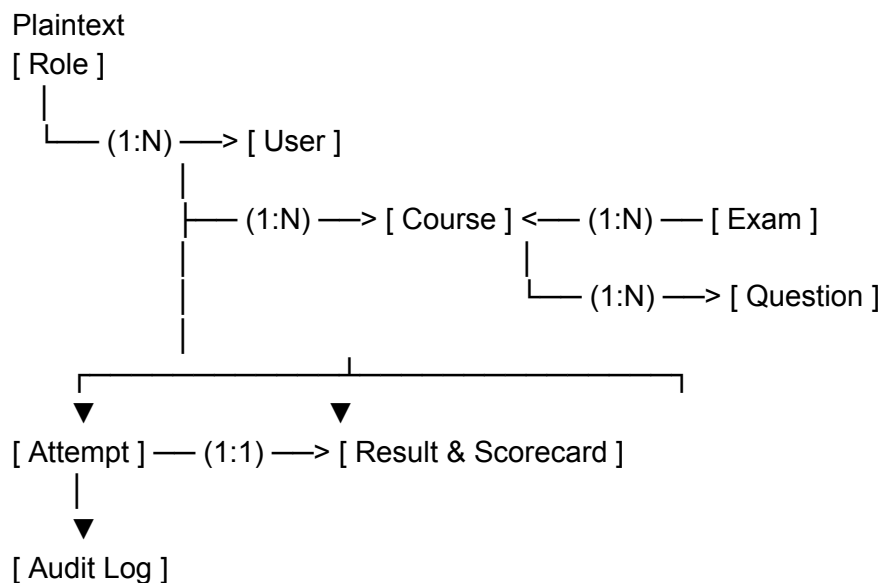
- **Session Management:** Core logic explicitly prevents multiple active sessions for a single user account.

## 4. Database Design

The database schema is highly relational and normalized to maintain data integrity across the examination workflow.

### 4.1. Entity Relationship (ER) Flow

The following hierarchy demonstrates the core relationships between entities:



### 4.2. Core Entities

- **User & Role:** Manages authentication, profiling, and RBAC restrictions.
- **Course & Exam:** Central entities configuring exam duration, passing criteria, and negative marking rules.
- **Question & Option:** A comprehensive question bank supporting various difficulty levels, MCQ choices, and starter code for programming assessments.
- **Attempt:** Tracks precise **startedAt** and **submittedAt** timestamps for server-side timer validation.
- **Result:** Auto-generated ledger containing the final percentage, marks obtained, and Pass/Fail status.
- **AuditLog:** Records administrative actions and critical system modifications for traceability.

## 5. API Modules Summary

The backend exposes a comprehensive set of RESTful APIs, logically grouped into specific feature modules.

| Module        | Key Functionalities                                                           |
|---------------|-------------------------------------------------------------------------------|
| Auth          | User Registration, JWT Login generation, Password change, Profile retrieval.  |
| Users / Admin | Platform statistics dashboard, User listing, Account activation/deactivation. |
| Courses       | Instructor and Admin capabilities to structure academic subjects.             |
| Questions     | Creation of diverse assessment items (Coding formats and MCQs).               |
| Exams         | Configurations for time limits, max attempts, and question assignments.       |
| Attempts      | Secure exam initiation, answer submission, and historical attempt retrieval.  |
| Results       | Automated score fetching and instant PDF scorecard download endpoints.        |
| Leaderboard   | Real-time ranking retrieval based on top scores for specific exams.           |

|                   |                                                                    |
|-------------------|--------------------------------------------------------------------|
| <b>Audit Logs</b> | System-wide tracking of critical structural or user-state changes. |
|-------------------|--------------------------------------------------------------------|

## 6. Security Features & Implementation

Security is built into the foundation of this educational application. The following mechanisms directly satisfy strict technical requirements:

- **JWT Authentication:** Stateless and secure user session management.
- **Role-Based Access Control (RBAC):** Custom `@Roles()` Guards ensuring strict boundaries between Admins, Instructors, and Students.
- **Single Active Session Engine:** Logs out previous sessions if a user authenticates from a new device, preventing collaborative cheating.
- **Server-Side Timer Enforcement:** Prevents client-side manipulation by strictly calculating the time difference between attempt creation and submission.
- **Password Hashing:** Bcrypt implementation preventing raw credentials from being exposed in the database.
- **DTO Validation:** NestJS `ValidationPipe` strictly validates all incoming payload shapes and types before controller execution.

## 7. Evaluation Engine & Workflow

### 7.1. State Transitions

The assessment lifecycle is designed as a strict state machine:

PUBLISHED EXAM → ATTEMPT STARTED → ANSWERS SUBMITTED →  
AUTO-EVALUATED → RESULT PUBLISHED

### 7.2. Auto-Evaluation Engine

Upon submission, the `AttemptsService` instantly parses the payload:

- **Verification:** Cross-references submitted options with the correct answers in the database.
- **Scoring Logic:** Adds assigned marks for correct answers and deducts fractions based on the `negativeMarks` configuration for incorrect ones.
- **Finalization:** Calculates the total percentage, determines the Pass/Fail flag based on `passingMarks`, and immediately shifts the attempt status to `PUBLISHED`.

## 8. Testing & Deployment

- **Unit Testing:** The project utilizes Jest. Core modules (such as `AuthService`) are tested in isolation using mocked Prisma instances and JWT services to ensure business logic remains pristine.
- **Docker Setup:** A highly optimized `docker-compose.yml` is implemented to orchestrate the PostgreSQL database, eliminating manual local configuration.
- **Winston Implementation:** Ensures production-ready traceability by routing standard logs to the terminal and critical failures to a dedicated file system.

## 9. Conclusion

The developed Online Examination System Backend API satisfies all functional and technical requirements specified while strictly adhering to modern enterprise backend development practices. The combination of strict TypeScript typings, Prisma's reliable data mapping, auto-evaluation workflows, and NestJS's robust architecture results in a highly scalable and fault-tolerant educational engine.

## 10. Links & References

- **Source Code Repository:** 🖱️ [\[https://github.com/ankittrip/exam-system\]](https://github.com/ankittrip/exam-system)(<https://github.com/ankittrip/exam-system>)
- **Live API Documentation (Swagger):** 🖱️ <http://localhost:3000/api/docs> (*Requires local server to be running*)